AutoGrader: A Rule-Driven Evaluator for C Programs

Design and implementation report: what was built, where, and how

October 2026

Abstract

AutoGrader is an agent that takes a student's C submission and an instructor-written rules file, compiles and runs the program inside a sandbox with fixed time and memory limits, evaluates every metric in the rules with its own analyzer, and returns one weighted grade together with feedback. For an incorrect program, the feedback gives a concrete failing input with the expected output beside it, a sanitizer diagnosis and an explanation. The backend is Django; the frontend is plain HTML and CSS. Readability is judged by an LLM on Groq, and a locally running Ollama `qwen3:4b` generates adversarial test inputs, tutor-style explanations and AI-written reference solutions for copy detection. Originality is measured on an *abstract structural* representation of the code. The central contract (*same submission + same rules → same grade*) was verified experimentally: 9 sample submissions graded 3 times each gave identical grades, metric scores and flags.

Contents

1 Goal and the reproducibility contract **2**

1.1 Where each requirement is implemented 2

2 Architecture **3**

2.1 Project layout 3

3 The rules file **4**

4 Safe execution: the sandbox **5**

5 The coordinator **6**

5.1 Phase 1: first pass (cheap sanity sweep) 6

5.2 Phase 2: dispatch 6

5.3 Phase 3: collation and the grade 6

6 The analyzers **7**

6.1 `io_tests`: correctness (`io_tests.py`) 7

6.2 `compiler_warnings`: syntax (`compiler_warnings.py`) 7

6.3 `static_analysis`: structure (`static_analysis.py`) 7

6.4 `function_split`: modularity (`function_split.py`) 7

6.5 `edge_case_runner`: logic and counter-tests 7

6.6 `complexity_estimator`: time efficiency (`complexity_estimator.py`) 8

6.7 `sanitizer_run`: memory safety (`sanitizer_run.py`) . . . 8
6.8 `llm_review`: readability via Groq (`llm_review.py`) . . . 9
6.9 `similarity_flag`: originality (`similarity_flag.py`) . . . 9

7 LLM integration **9**

8 Originality: abstract structural fingerprints **10**

9 Reproducibility **11**

10 Feedback and reports **11**

11 The Django web application **12**
11.1 Model . . . 12
11.2 Services: storage and the grading worker . . . 12
11.3 Views and URLs . . . 13
11.4 Templates and styling . . . 13

12 Running unattended: CLI, command and scripts **13**

13 Sample submissions and results **14**

14 Usefulness and limitations **14**
14.1 Usefulness . . . 14
14.2 Limitations (each names a case with a wrong or unreliable verdict) . . . 15

15 Problems found during development and how they were fixed **16**

16 Running on Linux **16**

17 Setup **17**

A Function index **17**

1 Goal and the reproducibility contract

Grading programs by hand is slow, inconsistent and does not scale. The assignment asks for an agent that does the whole job unattended:

1. **run** the submission safely, under fixed time and memory limits, surviving programs that crash or never halt;
2. **read** the grading rules from a file kept *outside* the agent's source code;
3. **evaluate** each metric with its own analyzer and combine the results into one grade using the weights in the rules;
4. **produce feedback**, including a specific failing input with the expected output, memory-leak reports and profiling;
5. **flag** suspected copying from peers, online resources or LLMs;
6. state the **usefulness** and **limitations** of the agent with 3–4 concrete examples each.

The fixed requirement is the **contract**: the same submission judged under the same rules must receive the same grade every time. Section 9 describes how this is achieved even though LLMs and wall-clock timings are inherently noisy.

1.1 Where each requirement is implemented

Requirement	Implementation	Location
Safe execution, time/memory limits, crash and hang survival	Process-group sandbox, rlimits, memory polling, file-based I/O	`engine/sandbox.py` (`run`, `compile_c`)
Rules outside the code	JSON rules files, validated at load time and in the web editor	`rules/*.json`, `engine/agent.py` (`validate_rules`)
One analyzer per metric	Plug-in registry, 9 analyzers	`engine/analyzers/`
Weighted overall grade	Collation in rules order	`engine/agent.py` (`Agent._grade`)
Failing input + expected output	Hidden tests, generated corner cases, reference-solution oracle	`io_tests.py`, `edge_case_runner.py`, `engine/context.py`
Counter-test generation (corner cases)	Category generators + LLM-proposed adversarial inputs	`engine/inputs.py`, `edge_case_runner.py`
Memory leaks / UB (valgrind/asan)	ASan + UBSan; macOS `leaks`; LeakSanitizer/valgrind on Linux	`sanitizer_run.py`
Profiling	Per-test CPU, wall time, peak memory; scaling curve	`sandbox.py`, `complexity_estimator.py`
Plagiarism / LLM-copy flag	Abstract structural fingerprints, AI-generated corpus	`similarity_flag.py`, `engine/clexer.py`
Readability via Groq	Rubric prompt, JSON output, cached	`llm_review.py`, `engine/llm.py`

Requirement	Implementation	Location
Ollama `qwen3:4b`	Counter-tests, tutor explanations, AI reference solutions	`edge_case_runner.py`, `agent.py`, `similarity_flag.py`
Usefulness & limitations	Included in every report	`engine/usefulness_limitations.json`
Django backend, HTML/CSS frontend	App with worker, views, templates, one stylesheet	`grader/`, `autograder_site/`
Unattended end-to-end run	CLI, management command, HTTP API	`engine/__main__.py`, `grader/management/commands/grade.py`

2 Architecture

The system is split into a **framework-independent engine** (`engine/`) and a **Django web application** (`grader/`). The engine can be used on its own from the command line; Django only stores submissions, queues work and renders reports.

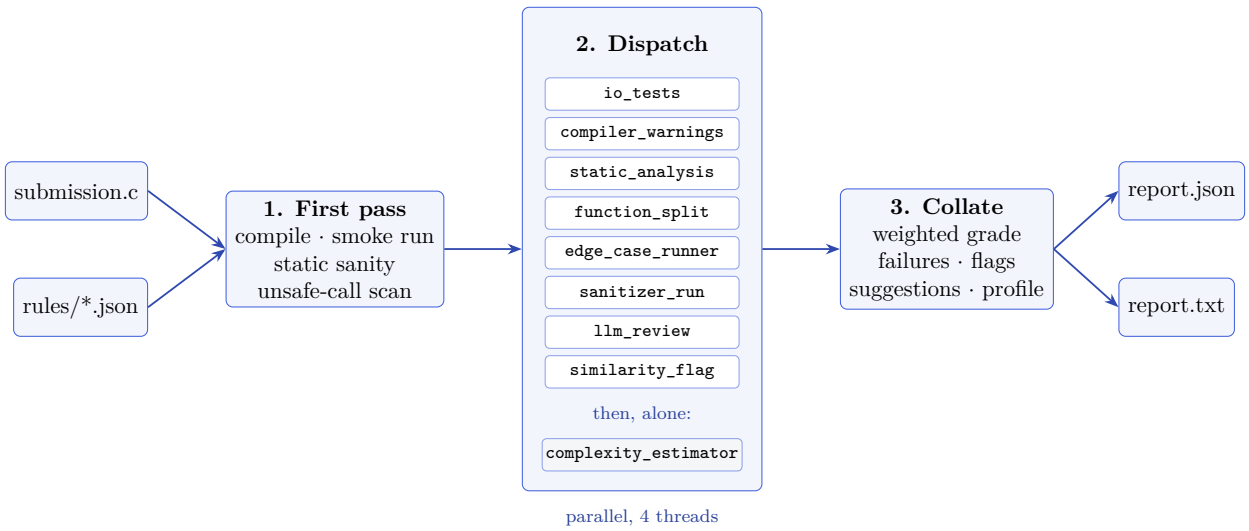


Figure 1: Coordinator pipeline implemented in `engine/agent.py`.

2.1 Project layout

```
autograder/
  engine/                  framework-independent agent
    agent.py               coordinator: rules validation, first pass, dispatch, collation
    sandbox.py             compile + run under time / memory / output limits
    context.py             shared state: binaries, reference oracle, tests, seeded RNG
    inputs.py              input_format -> deterministic edge-case generation + validation
    clexer.py              C lexer, abstract token stream, control skeleton, loop depth
    llm.py                 Groq + Ollama clients behind a content-addressed cache
    explain.py             deterministic "WHY" explanations + sanitizer root-cause hints
    sanitizers.py          ASan / LSan / UBSan report parser (clang and gcc)
    launcher.c             Linux run launcher (exact rusage, rlimits in C)
    report.py              plain-text student report
    usefulness_limitations.json
    analyzers/             base.py + one module per analyzer (plug-ins)
    __main__.py            CLI: python -m engine
```

```
grader/                 Django app: models, services (worker), views, templates, CSS
  management/commands/grade.py
autograder_site/        Django settings and root URLs
rules/                  instructor rules (add_two_integers.json, sum_array.json)
problems/               statements + reference solutions
tests/                  hidden tests (NN.in / NN.out)
submissions/            sample submissions; web uploads in submissions/<problem>/uploads/
corpus/llm_generated/   qwen3-generated solutions used for AI-copy detection
reports/                generated JSON + text reports
scripts/check_reproducibility.py, scripts/setup_linux.sh
Dockerfile, docker-compose.yml   Linux image (Ubuntu 24.04, gcc 13)
.cache/llm/             cached LLM responses (part of the grading record)
```

3 The rules file

Rules are plain JSON in `rules/`. They contain everything the agent needs, so adding a metric, changing a weight or a threshold, or defining a new problem never requires touching the engine. `rules/add_two_integers.json` reproduces the assignment's example and adds the fields the agent needs in order to *generate* its own tests:

```
{
  "problem": "add_two_integers",
  "statement_file": "problems/add_two_integers/statement.md",
  "reference_solution": "problems/add_two_integers/reference.c",
  "time_limit_s": 1,
  "memory_limit_mb": 64,
  "compile_flags": "-O2 -Wall -Wextra -std=c11",
  "compare": "tokens",
  "sample_input": "7 5\n", "sample_output": "12\n",
  "input_format": {"lines": [[{"name": "a", "type": "int"}, {"name": "b", "type": "int"
    }]]},
  "metrics": [
    {"name": "correctness", "analyzer": "io_tests", "weight": 0.20,
     "params": {"testset_dir": "tests/add/"}},
    {"name": "logical", "analyzer": "edge_case_runner", "weight": 0.10,
     "params": {"cases": ["negatives", "zero", "int_overflow", "whitespace", "llm"]}},
    {"name": "readability", "analyzer": "llm_review", "weight": 0.10,
     "params": {"model": "openai/gpt-oss-120b", "temperature": 0, "seed": 42}},
    ...
  ]
}
```

Key	Meaning
`problem`, `statement_file`	Name and problem text. The text is given to the LLM analyzers.
`reference_solution`	Instructor's C solution, used as the *oracle* that produces the expected output for generated inputs.
`time_limit_s`, `memory_limit_mb`	Sandbox limits for each run.
`input_format`	Typed stdin layout: lines of fields with `type` (`int`, `long`, `uint`, `float`, `word`), optional `min`/`max`, and `repeat` for arrays sized by another field.
`compare`	Output comparison: `tokens` (default, whitespace-insensitive), `lines`, `exact`, `float:1e-6`.
`metrics[]`	`name`, `analyzer`, `weight`, `params`. Weights are normalised if they do not sum to 1.

Validation. `validate_rules` in `engine/agent.py` rejects a missing or empty metric list, unknown analyzers (listing the available ones), duplicate metric names, negative or non-numeric weights, non-object params and all-zero weights. The same function runs when the web editor saves a file, so an invalid rules file can never be stored through the UI.

A second rules file, `rules/sum_array.json`, defines a harder problem (sum of $n \leq 10^6$ integers, required $O(n)$) so that complexity, leak and copy detection have something real to detect.

4 Safe execution: the sandbox

`engine/sandbox.py` compiles and runs untrusted code. `compile_c` invokes the system compiler (`cc`: Apple clang on macOS, gcc on most Linux distributions; it falls back to `gcc` or `clang`, and `AUTOGRADER_CC` overrides it) and parses every `file:line:col: warning|error: message [-Wflag]` diagnostic into structured data.

`run` executes a binary with these guarantees:

- **Time limit = CPU time** (user + sys), as on competitive-programming judges, so machine load cannot turn a correct program into a timeout. `RLIMIT_CPU` kills spinning programs; a wall-clock *backstop* of 3× the limit + 1 s kills programs that hang without using CPU. The child runs in its own session, and the whole process group is killed with `SIGKILL`, so forked children die too.
- **Memory limit** on *resident* memory. macOS ignores `RLIMIT_AS` (we verified that `setrlimit` fails), so the parent polls the child's physical footprint every 2–10 ms via `proc_pid_rusage` in `libproc` (class `_MacFootprint`). On Linux it polls `VmRSS` in `/proc/<pid>/status`, with `RLIMIT_AS` at 2× limit + 64 MB as a safety net. A cap at the limit itself would make `malloc` fail on untouched virtual reservations and turn an MLE into a crash. The program's `ru_maxrss` is checked after the run as a second line of defence.
- **Output limit:** `RLIMIT_FSIZE` on the redirected output (`SIGXFSZ` → OLE).
- **No core dumps**, a private temporary working directory, and a scrubbed environment (`PATH=/usr/bin:/bin`, `LANG=C`).
- **Files instead of pipes** for stdin/stdout/stderr, so a program that floods its output cannot deadlock the grader.

Each run returns a `RunResult`: stdout, stderr, exit code, signal name, flags for timeout, memory and output limits, wall time, CPU time (user+sys from `os.wait4`) and peak memory. Its `verdict` property maps these onto the judge verdicts `OK`, `TLE`, `MLE`, `OLE` and `RE`.

Linux launcher. On Linux, a child forked from the grader inherits the grader's own memory peak in `ru_maxrss` across `exec`. In testing, a 1 MB program was reported at 30–40 MB, and once the long-running grader grew past 64 MB *every* test was marked "memory limit exceeded". On Linux, programs are therefore started through `engine/launcher.c`, a 60-line C program compiled once on first use (`_launcher`). It forks the student program from a tiny process, applies the rlimits in C (replacing Python's `preexec_fn`, which is not thread-safe while analyzers run in parallel), writes the program's pid for memory polling, and reports the program's own `wait4` status, CPU time and peak memory. With the grader holding 300 MB, a 1 MB program is now reported at 1.25 MB, a 100 MB allocation is killed as MLE and a 40 MB one passes.

Additionally, the first pass statically refuses to execute programs that call `system`, `fork`, `exec*`, `popen`, `socket`, `kill`, `unlink`, `remove` and similar (`DANGEROUS_CALLS` in `engine/clexer.py`).

5 The coordinator

`Agent.grade` in `engine/agent.py` creates a private work directory, builds a `Context` (`engine/context.py`) shared by all analyzers, and runs three phases.

5.1 Phase 1: first pass (cheap sanity sweep)

`Agent._first_pass`:

1. **Compile check** with the rules' `compile_flags` (default `-O2 -Wall -Wextra -std=c11`).
2. **Smoke run** on the sample input under the rule limits.
3. **Static sanity:** `main` present, an input call (`scanf`, `getchar`, `fgets` ...), an output call (`printf`, `puts` ...), `return` inside `main`, and the unsafe-call scan.

If the smoke run times out, the report gets a `suspected_infinite_loop` flag; the execution-based analyzers (correctness, logical, complexity, memory safety) are *skipped with score 0*, while the source-only analyzers (syntax, structure, modularity, readability, originality) still run, producing a partial report, exactly as described in the assignment. A compile failure or an unsafe call has the same effect.

5.2 Phase 2: dispatch

Every metric is mapped to its analyzer class through the registry. Analyzers that need a binary are skipped if the first pass failed. The rest run in a `ThreadPoolExecutor` with four workers; analyzers marked `timing_sensitive` (the complexity estimator) run *alone afterwards*, so parallel work does not distort CPU timings. An exception inside an analyzer is caught and turned into a score of 0 with the error as evidence, so one faulty analyzer cannot destroy the report.

5.3 Phase 3: collation and the grade

Results are collected *in rules order*, never in completion order. With normalised weights w_i and analyzer scores $s_i \in [0, 1]$:

$$\text{grade} \;=\; 100 \cdot \frac{\sum_i w_i \, s_i}{\sum_i w_i}.$$

Collation also merges and de-duplicates failures from all analyzers (keyed by input), adds the AI-tutor explanations (Section 7), ranks suggestions by the points the corresponding metric lost, builds a profile from the per-test timings and the scaling curve, and records hashes and tool versions.

6 The analyzers

Analyzers subclass `Analyzer` in `engine/analyzers/base.py`, declare `needs_binary` and `timing_sensitive`, and return a `Result` with a score in $[0, 1]$, a feedback string, raw evidence, failures, suggestions, flags and a one-line verdict. The `@register` decorator adds them to `REGISTRY`, which is how rules refer to them by name.

6.1 `io_tests`: correctness (`io_tests.py`)

Runs every `NN.in` in `testset_dir`, compares the output with `NN.out` (or `.ans`, or the reference solution's output if no answer file exists), and records a verdict, CPU time, wall time and peak memory per test. Every failure is explained by `heuristic` in `engine/explain.py` and re-run under ASan+UBSan by `sanitizer_hint`, which returns the root cause, e.g. *runtime error at line 5: signed integer overflow: 2147483647 + 1 cannot be represented in type 'int'*. **Score:** passed / total.

6.2 `compiler_warnings`: syntax (`compiler_warnings.py`)

Compiles with the rule's flags, but with `-Werror` removed so warnings are counted individually instead of collapsing into one error. It reports whether the build *would* fail under `-Werror`. **Score:** $\max(0,\ 1 - 0.1 \cdot \#\text{distinct warnings})$, and 0 on compile errors.

6.3 `static_analysis`: structure (`static_analysis.py`)

Uses **lizard** for per-function cyclomatic complexity and length, compared with `max_cyclomatic` and `max_function_length`, plus deterministic token-level checks: comments and a header comment, single-block code, mutable globals, `goto`, long lines, magic numbers, and `cppcheck` findings if `cppcheck` is installed. **Score:** $1 - \sum$ deductions with 0.2 per over-complex or over-long function, 0.2 for no comments (0.1 for a missing header comment), 0.1 for single-block code, 0.1 per global (max 0.2), 0.1 for `goto`, 0.05 for long lines, 0.05 for ≥ 3 magic numbers, 0.05 per cppcheck issue (max 4).

6.4 `function_split`: modularity (`function_split.py`)

Counts functions (lizard), helpers other than `main`, the share of code lines in `main`, and helpers that are never called. **Score:** 1 if $n \geq$ `min_functions`, else $0.3 + 0.7 \min(1, h/(\texttt{min} - 1))$; capped at 0.3 when helpers are expected but absent (so `add.c` gets 3/10, as in the assignment); -0.2 if `main` holds more than 75 % of the code; -0.1 for unused helpers.

6.5 `edge_case_runner`: logic and counter-tests

This analyzer **generates counter test cases**. `InputFormat.category` in `engine/inputs.py` builds deterministic inputs per category from the `input_format`:

`zero`	all values 0; zero mixed with other values; the minimum size
`negatives`	-1, small and large negatives (only if the format allows them)
`int_overflow`	patterns [max], [min], [max, 1], [min, -1], [max, min] for full-range integer fields
`boundaries`	every field at its declared minimum and maximum; minimum and maximum sizes
`random`	seeded random instances, small and full range
`whitespace`	the same data with tabs, multiple spaces, blank lines, missing trailing newline
`large`, `single`, `empty`	maximum size, $n = 1$, $n = 0$
`llm`	`qwen3:4b` reads the statement *and the student's code* and proposes inputs likely to break *this* program

The random generator for each category is seeded from `sha256(rules_hash + category)` (`Context.rng`), so every run produces byte-identical inputs. LLM-proposed inputs are validated by `InputFormat.validate` (token count, types, ranges) before use. This was necessary because qwen3 often proposed a single number for a two-integer format. Expected outputs come from the compiled **reference solution** (`Context.expected`); inputs the reference rejects are not counted. Without a reference solution the analyzer checks robustness only (no crash, hang or sanitizer report). **Score:** passed / total, with per-category breakdown and targeted suggestions.

6.6 `complexity_estimator`: time efficiency (`complexity_estimator.py`)

Empirical part. For each size n in `sizes`, an input is generated with the size field set to n (`InputFormat.sized_input`). The program runs `repeats` times and the **minimum CPU time** is kept (the minimum is the most noise-robust estimator). The smallest size is used as the baseline, and a least-squares slope of $\log(t_n - t_0)$ against $\log n$ is fitted, ignoring points below a 4 ms noise floor. Slope bands map to classes: $< 0.35 \to O(1)/O(\log n)$, $< 1.35 \to O(n)/O(n \log n)$, $< 2.4 \to O(n^2)$, otherwise $O(n^3)$. A timeout at some size means the bound is exceeded; a crash at some size discards that and larger sizes.

Static part. `max_loop_depth` in `engine/clexer.py` computes the maximum loop nesting (handling braced and single-statement bodies and `do`–`while`); self-calls inside a function body mark recursion; `qsort` implies at least $O(n \log n)$. The static estimate breaks ties between neighbouring classes and is used when timings are flat. For inputs without a size field (such as $a + b$), flat timings mean $O(1)$.

Score: with class ranks r, $d = r_{\text{measured}} - r_{\text{required}}$; score 1 if $d \leq 0$, else $\max(0, 1 - 0.5d)$.

6.7 `sanitizer_run`: memory safety (`sanitizer_run.py`)

Builds with `-fsanitize=address,undefined -g -fno-omit-frame-pointer` and runs every hidden test (optionally the generated edge inputs too). It parses UBSan `runtime error` lines and ASan `ERROR: AddressSanitizer: <kind>` with the `SUMMARY` location, and collects distinct *(kind, line)* issues: heap/stack/global buffer overflow, use-after-free, double free, undefined behaviour.

All sanitizer output goes through one parser, `engine/sanitizers.py`. It takes the source line from the first stack frame inside the submission (`main.c:LINE`) rather than from the SUMMARY line, which with gcc points into the sanitizer runtime (for example, the `scanf` interceptor). LeakSanitizer reports are mapped to the same `memory-leak` kind as the macOS tool.

Leaks: LeakSanitizer is not supported on macOS, so on macOS the debug build runs under `leaks -atExit` with `MallocStackLogging=1`, which reports the leaked bytes *and the source line*. On Linux, LeakSanitizer runs inside the ASan build, or valgrind (`definitely lost`) is used if

installed; both report the leak and its line. LSan needs `ptrace`, which some containers forbid; the report then says the leak check was unavailable instead of claiming there are no leaks.

Self-test. Before trusting the sanitizers, `sanitizers.self_test` builds and runs two tiny programs with a known heap overflow and a known signed overflow, and checks that both are reported. ASan can be installed yet unusable: under x86_64 emulation its shadow-memory reservation got the process killed, and some kernels with high address randomisation crash older ASan runtimes. Without the check, this looked like "no diagnostics" and gave a false 10/10. If the self-test fails, the analyzer returns `unavailable`: the metric is left out of the grade, the remaining weights are renormalised, and a `metric_unavailable` flag explains why. A student is neither rewarded nor penalised for the grading machine. After two timeouts it stops early, so a hanging program does not spend minutes under sanitizers. **Score:** $\max(0, 1 - 0.25 \cdot \#\text{distinct issues})$.

6.8 `llm_review`: readability via Groq (`llm_review.py`)

The code is sent with line numbers and a fixed 0–10 rubric (names, comments, formatting, consistency, clarity; correctness explicitly excluded) to Groq, with temperature 0, seed 42, JSON output and 2000 tokens. The reply `{"score", "reasons", "suggestions"}` becomes the score (score/10), the feedback and line-specific suggestions; the raw response is kept as evidence.

Model choice. The assignment's example names `llama-3.3-70b-versatile`, but Groq no longer offers it on this account (HTTP 404, `model_not_found`). The available models were compared on `add.c`: `openai/gpt-oss-120b` gave 5/10 with accurate, line-specific suggestions; `qwen/qwen3.8-27b` also gave 5/10, but claimed a space was missing when it was not. `openai/gpt-oss-120b` was chosen. It matches the assignment's expected 5/10 for `add.c`.

Fallback. Without `GROQ_API_KEY` or network access, a clearly labelled deterministic heuristic (single-letter names, comment density, cramped spacing, indentation consistency) is used.

6.9 `similarity_flag`: originality (`similarity_flag.py`)

Described in detail in Section 8.

7 LLM integration

`engine/llm.py` contains `GroqClient` and `OllamaClient`, both behind the content-addressed `LLMCache`. `load_dotenv` reads `GROQ_API_KEY` from `.env`.

Use	How
Readability (Groq)	`openai/gpt-oss-120b`, T=0, seed 42, JSON object mode (`llm_review.py`).
Adversarial counter-tests (Ollama)	qwen3 gets statement, input format and code; returns `{"inputs": [{"input","why"}]}` (`EdgeCaseRunner._llm_inputs`).
AI-tutor explanations (Ollama)	One batched call explains the top three failures as `{"case","root_cause","fix"}` (`Agent._llm_explain`).
AI reference solutions (Ollama)	Three solutions (seeds 1–3, T=0.7) written once per problem to `corpus/llm_generated/<problem>/` (`SimilarityFlag._llm_solutions`).

Cache first. Every client checks the cache *before* checking whether its service is reachable. A grading machine without Ollama, such as a Linux server, therefore reproduces the grades made on a machine with it, provided `.cache/llm` is copied along.

Problem found: qwen3 keeps thinking. The installed `qwen3:4b` produced long reasoning text in its answer even with Ollama's `think: false`, with the `/no_think` soft switch, and with an empty `<think></think>` pre-fill in raw mode. A single code-generation call took 70 s and returned no code. **Fix:** every Ollama request uses grammar-constrained **JSON mode** (`format: "json"`), which forces the model to answer directly. Code is requested as an *array of lines* (`{"lines": [...]}`), because escaping a whole program into one JSON string triggered Ollama's "token repeat limit". Each call then takes about 4 s.

8 Originality: abstract structural fingerprints

The originality check compares programs on an *abstract* representation ("abstract schematics") rather than on raw text, so the usual disguises do not hide a copy. All functions are in `engine/clexer.py`.

1. **Lexing** (`tokenize`): a regular-expression C lexer; comments, whitespace and preprocessor lines are dropped.
2. **Canonicalisation** (`canonicalize`): `for (A; B; C) S` is rewritten to `A; while (B) { S C; }`, so the loop style does not matter.
3. **Abstraction** (`abstract_tokens`): identifiers → `V`; numbers, strings and characters → `N`, `S`, `C`; `int/long/short/unsigned` → `INT`; `float/double` → `FLT`; `for/while/do` → `LOOP`; `++/-` → `INC`; compound assignments → `OPEQ`. Library calls (`scanf`, `malloc` ...) are kept because they carry meaning.
4. **Brace removal** (`similarity_stream`): `if (x) return 0;` and `if (x) { return 0; }` become identical.
5. **Two views:** token 5-grams, compared with Jaccard similarity J; and the control skeleton (`control_skeleton`: loops, branches, returns, library calls), compared with a sequence-alignment ratio S. Combined: $\text{sim} = 0.75\,J + 0.25\,S$.
6. **Base-code exclusion** (as in MOSS): 5-grams that also occur in the instructor's reference solution are removed before comparing, so shared boilerplate is not evidence of copying.
7. **Corpus:** every `.c` file under `corpus_dir`, which includes peers and web uploads, plus the qwen3-generated solutions, so "copied from an LLM" is detected the same way as peer copying. Byte-identical files (the same submission uploaded again) are excluded from scoring and reported as `exact_duplicate`.
8. **Size gate:** programs with fewer than 50 abstract tokens are reported but never flagged.

Score: 1 below the threshold θ (0.85); otherwise $1 - (\text{sim} - \theta)/(1 - \theta)$, together with a `suspected_copy` or `suspected_llm_copy` flag for human review.

Example. `sum_copy.c` renames every identifier of `sum_good.c`, replaces the `for` loop with a `while` loop, changes brace style and rewrites the comments. Both reduce to the same stream:

```
INT V ( INT V ) INT V = N ; INT V = N ; LOOP ( V < V ) INT V ; if ( scanf ( S , & V )
    != N )
break ; V OPEQ V ; V INC ; return V ; INT V ( void ) INT V ; if ( scanf ( S , & V ) !=
    N )
return N ; printf ( S , V ( V ) ) ; return N ;
```

Both submissions are flagged at similarity 1.00. Before canonicalisation and brace removal were added, the same pair scored only 0.57 and was missed.

Why the size gate exists. During testing, the assignment's 6-line `add.c` was flagged at 1.00 as an LLM copy, because qwen3 wrote exactly the same program. Short programs are written identically by almost everyone, so a match carries no evidence; the gate removes this false positive.

9 Reproducibility

- **Seeded generation:** all generated inputs come from `random.Random` seeded with `sha256(rules_hash + tag)`.
- **Deterministic order:** analyzer results are collected in rules order; every file and corpus listing is sorted.
- **LLM cache:** each request is keyed by `sha256(provider, model, messages, temperature, seed, json)` and stored in `.cache/llm/<key>.json`; later runs replay the first answer. Deterministic model-side failures (for example, an Ollama generation aborted by the repeat limit, or a Groq HTTP 400) are cached as well, so a later successful retry cannot change the grade. Network, authentication and rate-limit errors are never cached.
- **Stable timing:** time limits are CPU time, not wall-clock time; minimum-of-N CPU time is mapped onto discrete complexity classes; the timing-sensitive analyzer runs alone; the web worker grades one submission at a time. CPU-based limits were added after a reproducibility failure: a freshly compiled binary took 0.42 s wall-clock for 0.01 s of CPU on its first launch (macOS scans new binaries), and under load a correct `add.c` once exceeded the 1 s wall-clock limit and was flagged as an infinite loop.
- **Audit trail:** each report stores `submission_sha256`, `rules_hash`, `inputs_hash` (rules + tests + reference), the corpus hash, tool versions and the engine version.

Why the cache is necessary. Three fresh Groq calls with temperature 0 and seed 42 scored `sum_leak.c` 7, 8 and 7 out of 10. Without the cache, the overall grade would drift by 0.5 points between runs.

Experiment. `scripts/check_reproducibility.py` grades every sample several times and compares the grade, every metric score and the set of flags. All 9 samples $\times$ 3 runs were identical on macOS and on Linux, including a run where both platforms were grading at the same time to load the machine.

10 Feedback and reports

Every grading produces two files, written by the CLI and by the web worker:

- `reports/<name>.json` for machines: first pass, every metric with weight, score, contribution, verdict, feedback and full evidence; failures; flags; suggestions; profile; usefulness and limitations; hashes; tool versions.
- `reports/<name>.txt` for the student, rendered by `render_text` in `engine/report.py`.

Excerpt of `reports/add.txt` for the assignment's example submission:

```
===== Grading report for add.c =====
Overall grade: 75.7 / 100
Metric          Weight    Score  Contrib   Verdict
```

```
correctness          20    16/20     16.0   8 of 10 tests pass
syntax               10    10/10     10.0   clean
structure            10     7/10      7.0   No comments at all; All logic is in a single
     block
modularity           10     3/10      3.0   only main() is defined
logical              10   7.2/10      7.2   int_overflow not handled, llm not handled
complexity           10    10/10     10.0   O(1) confirmed (required O(1))
memory_safety        10   7.5/10      7.5   undefined-behaviour
readability          10     5/10      5.0   single-letter variable names (a,b), no
    comments ...
originality          10    10/10     10.0   too little unique code to judge
--- Where you failed ---
Test 07  (correctness, WA)
   INPUT:     2147483647 1
   EXPECTED:  2147483648
   GOT:       -2147483648  [exit 0]
   SANITIZER: runtime error at line 5: signed integer overflow: 2147483647 + 1 cannot
    be
              represented in type 'int'
   WHY: The correct result does not fit in a 32-bit int, so the computation wrapped
    around
   (signed integer overflow is undefined behaviour in C). Use a wider type such as long
     long.
   AI TUTOR: The sum of 2147483647 and 1 exceeds the maximum value for a 32-bit signed
   integer, causing an overflow. Fix: Use a 64-bit integer type for the sum.
```

The text report continues with memory/UB diagnostics, the profile (maximum CPU and wall time, peak memory, scaling table), ranked suggestions and the usefulness and limitations section.

11 The Django web application

11.1 Model

`grader/models.py` defines one model, `Submission`: student, problem, rules file, file name and path, source and its SHA-256, status (`queued`, `running`, `done`, `error`), grade, the full JSON report, the text report, error text and timestamps. SQLite is the database.

11.2 Services: storage and the grading worker

`grader/services.py` connects Django and the engine:

- `list_rules`, `read_rules`, `write_rules`, `safe_rules_name`: rules files are addressed by bare file name only (no path traversal) and validated before saving.
- `create_submission`: size limit (256 KB), stores the source in `submissions/<problem>/uploads/NNNNN_name.c`, which places it in the originality corpus of its problem.
- `enqueue` / `_work`: a single daemon worker thread consumes a queue, so submissions are graded one at a time. `grade_now` runs the agent, stores the report and also writes the JSON and text reports into `reports/`.
- `clear_all`: deletes every submission, its uploaded file and its reports, but keeps sample submissions, rules, tests and the LLM cache. A submission deleted while queued is skipped by the worker.

11.3 Views and URLs

URL	View	Purpose
/	index	Upload one or more .c files (drag and drop) or paste code; choose the rules.
/submissions/	history	History table, statistics (count, graded, average, flagged), problem filter.
/submissions/clear/	clear_submissions	POST only, confirmed in the browser; clears all submissions.
/submissions/<id>/	submission	Full report; polls the status endpoint while grading.
/submissions/<id>/status/	submission_status	JSON status for polling.
/submissions/<id>/regrade/	regrade	Re-runs the evaluation.
/submissions/<id>/report.json|txt	submission_download	Download reports.
/rules/, /rules/new/, /rules/<name>/	rules_list, rules_edit	List, create and edit rules with validation.
/api/grade/	api_grade	POST a file and a rules name, receive the JSON report synchronously.

11.4 Templates and styling

The templates in grader/templates/grader/ are base.html, index.html, submission.html, history.html, rules_list.html and rules_edit.html. All styling is one hand-written stylesheet, grader/static/grader/style.css, with light and dark themes via prefers-color-scheme and a responsive layout down to phone width; JavaScript is limited to small inline scripts (drag and drop, status polling, live JSON and weight checking in the rules editor, the clear confirmation).

The report page shows a grade ring, first-pass checks, flags, a metrics table with score bars and expandable evidence, failure cards (input / expected / got, sanitizer, why, AI tutor), suggestions, the profile, the source code with the lines that diagnostics point to highlighted (_highlight_lines in grader/views.py), the usefulness and limitations section and a reproducibility block with all hashes.

12 Running unattended: CLI, command and scripts

```
# engine only, no database
.venv/bin/python -m engine submissions/add_two_integers/add.c \
    --rules rules/add_two_integers.json --out reports/ [--offline]

# Django management command (batch); --save also records them in the web UI
.venv/bin/python manage.py grade submissions/sum_array/*.c --rules rules/sum_array.json
     --save

# HTTP API
curl -F file=@add.c -F rules=add_two_integers.json http://localhost:8000/api/grade/

# reproducibility check
.venv/bin/python scripts/check_reproducibility.py --runs 3

# web UI
```

```
.venv/bin/python manage.py runserver 8000
```

13 Sample submissions and results

Each sample has a deliberate flaw. The results below are produced by the agent with Groq readability enabled, and are identical across repeated runs.

Submission	Flaw	Grade	What the agent reported
`add.c`	assignment's example: `int` sum, all in `main`, no comments	75.72	8/10 tests (fails INT_MAX+1 and INT_MIN−1), UB at line 5, only `main`, readability 5/10
`add_clean.c`	none: 64-bit sum, helper, comments	100.00	everything passes
`add_infinite.c`	waits for a `'.'` that never arrives	40.00	`suspected_infinite_loop`; execution analyzers skipped, partial report
`add_syntax_error.c`	missing `;`	26.00	`compile_failed`; source-only analyzers still graded
`sum_good.c`	none	89.50	`suspected_copy` (1.00 vs `sum_copy.c`)
`sum_copy.c`	renamed, `for`→`while`, restyled copy	88.00	`suspected_copy` (1.00 vs `sum_good.c`)
`sum_ai.c`	qwen3's solution, renamed and re-commented	85.50	`suspected_llm_copy` (1.00 vs a qwen3-generated solution)
`sum_quadratic.c`	correct but $O(n^2)$	79.50	$O(n^2)$ exceeds $O(n)$; timed out at $n = 320\,000$
`sum_leak.c`	`int` accumulator, `malloc` never freed	77.55	5/8 tests, memory leak at line 12 (macOS `leaks`), UB at line 14
`sum_fixed_buffer.c`	`int buf[100]` overflows for $n > 100$	82.64	6/8 tests, stack-buffer-overflow at line 6

14 Usefulness and limitations

This section is also embedded in every report (`engine/usefulness_limitations.json`).

14.1 Usefulness

1. **Large programming courses (CS1 labs).** A 300-student lab is batch-graded with one command. Every student receives the same rubric and the exact failing input (`2147483647 1` → expected `2147483648`) instead of a bare mark.

2. **Coding-interview screens.** For "sum an array in $O(n)$", an $O(n^2)$ nested loop is separated by the timing curve (`sum_quadratic.c` times out at $n = 320\,000$) and an `int` accumulator by the `int_overflow` corner cases; every rejection is justified metric by metric.

3. **Plagiarism and LLM-copy triage.** Renamed, reformatted, `for`→`while` copies are flagged at 1.00, as are matches with qwen3-generated solutions; instructors review only the flagged pairs.

4. **Self-paced practice.** A student receives *memory-leak at line 12*, a sanitizer trace and an AI-tutor explanation at any hour, without staff time.

14.2 Limitations (each names a case with a wrong or unreliable verdict)

1. **Tests cannot prove correctness.** A program that hard-codes the hidden tests, or fails only for $n = 99\,991$, receives full correctness marks.

2. **Timing-based complexity is unreliable for small sizes.** With sizes $[1, 10, 100]$ an $O(n^2)$ program runs in under 1 ms everywhere; the static fallback then counts `for (i = 0; i < 3; i++)` as an $O(n)$ loop and treats a recursive merge sort (two self-calls) as possibly exponential.

3. **Similarity cannot judge tiny programs and misses semantic rewrites.** A copied 6-line $a + b$ receives full originality marks because of the size gate, and a copy rewritten from a loop into recursion is not flagged.

4. **Verdicts depend on the compiler and platform.** `while (b != 0) { s += 1; }` never terminates, but clang `-O2` may delete the side-effect-free loop (C11 §6.8.5p6). This was observed: the smoke run finished in 6 ms and the agent reported a wrong answer instead of an infinite loop. On macOS the memory cap is enforced by polling, so a 200 MB burst that ends within about 5 ms can pass a 64 MB limit.

An additional caveat is that LLM-based readability depends on the model: the cache makes re-grades identical, but clearing it or switching models (as was forced when Groq retired `llama-3.3-70b-versatile`) can change the readability score for the same code.

15 Problems found during development and how they were fixed

Problem	Fix
macOS ignores `RLIMIT_AS`	Poll the physical footprint through `libproc`; check `ru_maxrss` after the run.
LeakSanitizer unavailable on macOS	Use the system `leaks -atExit` tool with stack logging (reports the line).
clang `-O2` removed an infinite loop	Documented as a limitation; the sample was changed to a realistic hang on `getchar()` at EOF.
qwen3 reasons despite `think: false`	Grammar-constrained JSON mode for every Ollama call; code requested as an array of lines.
LLM inputs did not match the format	Validate every proposed input against `input_format`.
Renamed copy scored only 0.57	Canonicalise `for` into `while` and drop braces; the pair now scores 1.00.
6-line `add.c` flagged as LLM copy	Minimum-size gate (50 abstract tokens) before flagging.
A function's own name counted as recursion	Search for self-calls only inside the function body.
Timings taken from crashing runs	Record `crashed_at` and ignore those sizes.
Uncached model-side LLM failures	Cache deterministic failures so a later retry cannot change the grade.
Groq retired `llama-3.3-70b-versatile`	Compared available models; switched to `openai/gpt-oss-120b`.
Groq not deterministic at T=0	Confirmed (7, 8, 7); the content-addressed cache guarantees reproducibility.
Linux: every test "memory limit exceeded"	`ru_maxrss` inherited from the forked grader; programs are started by a C launcher.
Linux: leaks and ASan errors without a line	Shared parser takes the line from the first `main.c` stack frame.
Cached LLM answers unused without Ollama	Check the cache before checking whether the service is reachable.
Correct program flagged as an infinite loop under load	Time limits on CPU time with a wall-clock backstop.
False 10/10 memory safety where ASan cannot run (x86_64 emulation)	Sanitizer self-test; metric marked unavailable and left out of the grade.

16 Running on Linux

The agent was tested in containers, each installed from scratch with `scripts/setup_linux.sh`, on Ubuntu 22.04 (Python 3.10, gcc 11.4), Debian 12 (Python 3.11, gcc 12.2), Ubuntu 24.04 (Python 3.12, gcc 13.3) and Fedora 41 (Python 3.13, gcc 14.3) on arm64, and on Ubuntu 24.04 on x86_64 under emulation. The reproducibility check passed on all three Ubuntu/Debian systems with identical grades, which equal the macOS grades except where the platform itself differs (see below).

```
./scripts/setup_linux.sh                 # apt / dnf / pacman: gcc, libasan, libubsan,
    venv, database
WITH_VALGRIND=1 ./scripts/setup_linux.sh  # optional: valgrind instead of LeakSanitizer
.venv/bin/python manage.py runserver 0.0.0.0:8000
```

```
docker compose up --build                # Ubuntu image; project folder mounted at /
    app
```

	macOS	**Linux**
Compiler	Apple clang (`cc`)	gcc (`cc`), falls back to `gcc`/`clang`
Memory limit	footprint polling via `libproc`	`VmRSS` polling, `RLIMIT_AS` safety net
Process start	Python `preexec_fn`	C launcher (`engine/launcher.c`)
Leak check	`leaks -atExit`	LeakSanitizer, or valgrind if installed
Containers	n/a	`SYS_PTRACE` capability for LeakSanitizer (set in `docker-compose.yml`)

What the Linux test found. Running the unchanged code on Ubuntu exposed four problems: false memory-limit failures from the inherited `ru_maxrss` (fixed by the launcher); leaks reported as kind "detected" without a line, and ASan errors without a line (fixed by the shared parser); and cached LLM answers being ignored when Ollama was unreachable (fixed by checking the cache first). After the fixes, the reproducibility check passed on Linux, and the setup script was verified on clean Ubuntu and Fedora containers. The web UI and the HTTP API were tested in the container; `docker-compose.yml` itself was not run, because no compose tool is installed on the test machine.

Platform differences. Time limits and memory layout are properties of the machine. `sum_quadratic.c` ($O(n^2)$) is close to the time limit on hidden test 08 on a 2-CPU Linux VM but not on an M-series Mac, so its grade differs slightly. `sum_fixed_buffer.c` crashes at $n = 20\,000$ on Linux (smaller default stack) but not on macOS. Each machine is reproducible on its own; for a course, all submissions should be graded on one machine.

17 Setup

```
cd autograder
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt   # Django, lizard,
    requests
echo 'GROQ_API_KEY=...' > .env                                        # Groq key
ollama pull qwen3:4b                                                  # local model
.venv/bin/python manage.py migrate
.venv/bin/python manage.py runserver 8000
```

Requirements: a C compiler with `-fsanitize=address,undefined` (Apple clang or gcc). Optional: `cppcheck`, and `valgrind` on Linux. Tested with Python 3.14, Django 6.1, lizard 1.24, Apple clang 21 on macOS (arm64) and Ollama 0.40; and on Linux with Python 3.10–3.13 and gcc 11–14 (Ubuntu 22.04 and 24.04, Debian 12, Fedora 41; Section 16). Python 3.10 or newer is required.

A Function index

Function / class	**Role**
engine/agent.py	
`validate_rules`, `load_rules`	Parse and validate rules.
`Agent.grade`	Entry point: work directory, context, pipeline, text report.
`Agent._first_pass`	Compile, smoke run, static sanity, unsafe-call scan.

Function / class	Role
`Agent._grade`	Dispatch analyzers, collate weighted grade, failures, flags, suggestions.
`Agent._llm_explain`	Batched qwen3 explanations of the top failures.
`Agent._inputs_hash`	Hash of rules, tests and reference solution.
engine/sandbox.py	
`compile_c`	Compile and parse diagnostics.
`run`	Run under time, CPU, memory and output limits; returns `RunResult`.
`_MacFootprint`	macOS memory polling through `proc_pid_rusage`.
engine/context.py	
`Context.binary`	Lazily compiled variants: `main`, `sanitized`, `debug`.
`Context.expected`	Reference-solution oracle (cached per input).
`Context.test_cases`, `Context.edge_inputs`	Hidden tests; deterministic generated inputs.
`Context.rng`	RNG seeded from the rules hash.
`compare_output`	tokens / lines / exact / float comparison.
engine/inputs.py	
`InputFormat.category`	Edge-case generators per category.
`InputFormat.sized_input`	Inputs of exact size n for the complexity curve.
`InputFormat.validate`	Check LLM-proposed inputs against the format.
engine/clexer.py	
`tokenize`, `abstract_tokens`	Lexer and abstraction.
`canonicalize`, `similarity_stream`	for→while rewriting, brace-free stream.
`control_skeleton`, `kgrams`	Skeleton and fingerprints.
`max_loop_depth`, `called_names`	Static complexity helpers.
engine/llm.py	
`LLMCache`	Content-addressed response cache.
`GroqClient`, `OllamaClient`	Providers; `chat` returns (text, from_cache).
engine/explain.py, engine/report.py	
`heuristic`, `sanitizer_hint`	WHY text and sanitizer root cause for a failure.
`render_text`	Student-facing text report.
grader/	
`services.create_submission`, `enqueue`, `grade_now`	Storage and background grading.
`services.clear_all`	Delete all submissions, uploads and their reports.
`views.*`	Pages and API listed in Section 11.
`management/commands/grade.py`	Batch grading from the terminal.